

UNIVERSIDAD INTERNACIONAL DE LAS AMÉRICAS

ESCUELA DE INGENIERÍA INFORMÁTICA

PROYECTO

CURSO

**HERRAMIENTAS PARA EL DESARROLLO DE SISTEMAS DE
INFORMACIÓN**

**ABIGAIL ANDREA OBANDO MORA.
KEYLOR SEQUEIRA BADILLA
DIANA CHINCHILLA HERRERA**

PROFESOR

FEDERICO ANTONIO SANCHEZ DIAZ

SAN JOSÉ, COSTA RICA

Contenido

Introducción	5
Descripción del problema	5
Alcance y delimitación del proyecto	6
Objetivos	7
Objetivo General	7
Objetivo Especifico.....	7
Justificación	8
Marco Teórico.....	9
API REST.....	9
Protocolo HTTP.....	9
Base de datos relacional.....	10
Autenticación y segundo factor	10
Control de acceso basado en roles.....	11
Gestión de vuelos, asientos y reservas	11
Bitácora y trazabilidad	12
Notificaciones internas.....	12
Documentación de la API	13
Visualización de datos	13
Pruebas y depuración.....	13
Metodología	14
Selección y justificación de herramientas	16
Diseño del sistema	17
Arquitectura general.....	17
Roles del sistema.....	21

Matriz de permisos	21
Modelo de base de datos	23
Diagramas UML	24
<i>Diagrama general de casos de uso</i>	25
<i>Diagrama de secuencia de autenticación y OTP</i>	27
<i>Diagrama de secuencia de cambio de contraseña</i>	28
Endpoints principales	29
Descripción de módulos	39
<i>Módulo de autenticación</i>	39
<i>Módulo de Usuarios</i>	40
<i>Módulo de Aeronaves y Rutas</i>	40
<i>Módulo de Vuelos y Asientos</i>	41
<i>Módulo de Reservas</i>	41
<i>Módulo de Pagos</i>	41
<i>Módulo de Bitácora</i>	42
<i>Módulo de Notificaciones</i>	42
<i>Módulo de Reportes</i>	42
Relación entre los Módulos	43
Desarrollo	43
Conclusiones	43
Recomendaciones	43
Referencias	44
Anexo A	44
Anexo B	44

Anexo C	44
Anexo D	44
Anexo E	44

Introducción

El desarrollo de sistemas de información requiere una planificación ordenada, especialmente cuando se trabaja con procesos que involucran usuarios, disponibilidad, reservas, pagos y control de acciones dentro de una plataforma. A partir de esta necesidad, se propone el diseño y desarrollo de una solución web basada en una API REST para la gestión de vuelos y reservas, aplicando herramientas de ingeniería de software estudiadas durante el curso. El proyecto se desarrolla a partir del tema asignado sobre la creación de una API REST para la gestión de reservas de transporte. En este caso, la propuesta se orienta al contexto de reservas aéreas, con el propósito de construir una solución funcional que permita administrar el flujo principal relacionado con vuelos, asientos, reservas y pagos simulados.

La solución contempla módulos para usuarios, roles, autenticación con segundo factor académico, cambio de contraseña, bitácora de transacciones, notificaciones internas, aeronaves, rutas, vuelos, asientos, reservas, pagos simulados, comprobantes y reportes gráficos. Además, se utilizarán herramientas de modelado, desarrollo, pruebas, depuración, control de versiones y documentación, con el fin de evidenciar un proceso estructurado y coherente con los principios de ingeniería de software. Con este trabajo se busca demostrar el flujo principal de una reserva aérea, desde el inicio de sesión del usuario y la búsqueda de un vuelo disponible, hasta la selección de asiento, creación de la reserva, pago simulado, generación del comprobante y registro de las acciones realizadas. De esta manera, el desarrollo no se limita a mostrar pantallas o formularios aislados, sino que integra reglas de negocio, seguridad, trazabilidad y validaciones que permiten comprobar el comportamiento del sistema.

Descripción del problema

La gestión de vuelos y reservas implica controlar información que cambia constantemente, como la disponibilidad de asientos, el estado de los vuelos, los datos de los usuarios, las reservas realizadas y los pagos asociados. Cuando estos elementos no se administran de forma integrada, pueden generarse errores como duplicidad de reservas, pérdida de control sobre los asientos disponibles, dificultad para consultar datos actualizados y ausencia de evidencia sobre las acciones ejecutadas. Otro aspecto relevante es el control de acceso. En una plataforma de este tipo, no todos los usuarios deben tener los mismos permisos. El administrador necesita gestionar usuarios, aeronaves, rutas, vuelos, asientos, reservas generales, bitácoras y reportes, mientras que el cliente o pasajero debe limitarse a consultar vuelos disponibles, seleccionar asiento, crear una reserva,

realizar un pago simulado, consultar su comprobante y cancelar sus propias reservas cuando corresponda.

Sin una separación clara de roles, podrían presentarse accesos indebidos o exposición de información que no pertenece al usuario. También es necesario registrar las acciones relevantes realizadas dentro de la aplicación. Operaciones como el inicio de sesión, cambios de contraseña, programación de vuelos, creación de reservas, pagos simulados y cancelaciones deben quedar documentadas mediante una bitácora. Este registro permite dar seguimiento a los eventos, facilitar la depuración de errores y respaldar los resultados obtenidos durante las pruebas. Por esta razón, se propone desarrollar una API REST con una interfaz web básica que permita administrar el proceso de vuelos y reservas de forma ordenada, segura y verificable.

Alcance y delimitación del proyecto

El proyecto es orientado al desarrollo de una API REST para gestionar reservas de transporte. Aunque el enunciado contempla diferentes medios, como autobuses, trenes, aviones y transporte privado, este trabajo se delimita al transporte aéreo. Esta decisión permite mantener un alcance controlado, desarrollar una solución funcional dentro del plazo establecido y profundizar en el flujo de vuelos y reservas sin extender el sistema hacia una gestión multimodal. Dentro del alcance se incluye la administración de usuarios con roles definidos, autenticación con segundo factor académico, cambio de contraseña, bitácora de transacciones, notificaciones internas, gestión de aeronaves, rutas aéreas, vuelos programados, estados de vuelo.

Adicionalmente, asientos por vuelo, búsqueda de disponibilidad, creación de reservas, generación de código único, pago simulado, comprobante tipo boleto, cancelación de reservas, cancelación administrativa de vuelos, filtros administrativos y dashboard básico con apoyo de QuickChart o Chart.js. La delimitación establece que los vehículos serán tratados como aeronaves, las rutas y horarios como rutas aéreas y vuelos programados, y los asientos como espacios disponibles asociados a un vuelo específico. La selección de asientos se realizará mediante una visualización simple, sin representar un mapa realista de avión ni clases de cabina. El objetivo es demostrar la lógica de disponibilidad, selección y bloqueo de asientos, no reproducir la operación completa de una aerolínea comercial.

Quedan fuera del alcance la gestión de autobuses, trenes y transporte privado, así como escalas, conexiones, múltiples aerolíneas, reservas de varios pasajeros en una misma operación,

pasarelas reales de pago, reembolsos, facturación, recuperación automática de contraseña por correo, mapa complejo de aeronave y diseño comercial avanzado. Estas exclusiones permiten concentrar el desarrollo en los módulos principales, asegurar que las funcionalidades puedan probarse correctamente y presentar un producto final coherente con los requerimientos definidos.

Objetivos

Objetivo General

Desarrollar un sistema web basado en una API REST para la gestión de vuelos y reservas, que permita administrar usuarios, roles, autenticación, aeronaves, rutas aéreas, vuelos programados, asientos, reservas, pagos simulados, comprobantes, bitácoras, notificaciones y reportes gráficos, aplicando herramientas de análisis, modelado, programación, pruebas, depuración y documentación utilizadas durante el curso.

Objetivo Especifico

- Analizar los requerimientos funcionales y no funcionales del sistema, considerando el alcance definido para la gestión de vuelos y reservas, así como las limitaciones propias de un prototipo académico.
- Diseñar la estructura lógica del sistema mediante modelos, diagramas y definiciones técnicas que permitan representar los módulos principales, los roles de usuario, las reglas de negocio, la base de datos y el flujo general de operación.
- Implementar una API REST que permita el intercambio de información entre el backend y la interfaz web, utilizando endpoints organizados para usuarios, autenticación, aeronaves, rutas, vuelos, asientos, reservas, pagos, bitácoras, notificaciones y reportes.
- Desarrollar los módulos principales del sistema, incluyendo inicio de sesión, segundo factor de autenticación académico, cambio de contraseña, gestión de usuarios, administración de vuelos, selección de asientos, creación de reservas, pago simulado, generación de comprobante y consulta de información según el rol correspondiente.
- Incorporar mecanismos de control y trazabilidad que permitan registrar eventos relevantes mediante una bitácora, mostrar notificaciones internas y restringir el acceso a las funcionalidades de acuerdo con los permisos definidos para administradores y clientes o pasajeros.

- Integrar un reporte gráfico básico mediante QuickChart o Chart.js, con datos reales generados por el sistema, para visualizar información relacionada con reservas, ocupación de vuelos o estados registrados dentro de la plataforma.
- Realizar pruebas funcionales, de integración y depuración sobre los módulos desarrollados, verificando que el sistema cumpla con los requerimientos establecidos y que las operaciones principales se ejecuten de forma correcta, segura y coherente con el flujo de reservas aéreas.

Justificación

El desarrollo de este proyecto se justifica por la necesidad de construir una solución que permita representar, de forma práctica, el proceso de gestión de vuelos y reservas dentro de un sistema de información. Este tipo de proceso requiere controlar datos que dependen entre sí, como usuarios, vuelos, asientos, reservas y pagos simulados, por lo que resulta adecuado para aplicar análisis de requerimientos, diseño de base de datos, programación de servicios, pruebas y documentación técnica. La gestión de reservas aéreas permite trabajar con un flujo claro y verificable. Para que una reserva sea válida, el sistema debe comprobar la disponibilidad del vuelo, validar que el asiento no haya sido seleccionado previamente, asociar la operación con un usuario y actualizar el estado correspondiente según el resultado del pago simulado.

Esto permite evidenciar la importancia de las reglas de negocio y de la integridad de datos dentro de una aplicación. El proyecto también resulta pertinente desde el punto de vista técnico, ya que se desarrolla mediante una API REST consumida por una interfaz web. Esta arquitectura permite separar las responsabilidades del sistema, organizar los servicios por recurso y facilitar la validación de las funcionalidades mediante herramientas de prueba. Además, el uso de autenticación, roles y segundo factor académico permite incorporar controles básicos de seguridad acordes con el alcance definido. Otro elemento que justifica la propuesta es la trazabilidad. En un sistema donde se realizan acciones como iniciar sesión, crear reservas, modificar vuelos, simular pagos o cancelar operaciones, es necesario contar con registros que permitan revisar lo ocurrido.

Por ello, la bitácora de transacciones y las notificaciones internas aportan evidencia del comportamiento del sistema y sirven como apoyo para la depuración y validación de resultados. La inclusión de reportes gráficos básicos permite que los datos generados por la aplicación no

queden únicamente almacenados, sino que puedan interpretarse de forma visual. Esto aporta valor al módulo administrativo, ya que facilita la revisión de información relacionada con reservas, ocupación o estados registrados, sin convertir el sistema en una herramienta analítica compleja. Esta delimitación permite desarrollar un producto funcional, comprobable y coherente con los requerimientos establecidos, sin exceder el tiempo ni los recursos disponibles para el proyecto académico.

Marco Teórico

El desarrollo del sistema web de gestión de vuelos y reservas requiere comprender los conceptos técnicos que sustentan su funcionamiento. La arquitectura de la aplicación, la organización de los datos, el control de acceso y las pruebas no corresponden a elementos independientes, sino a decisiones que deben mantener coherencia con los requerimientos planteados. En este apartado se desarrollan únicamente los fundamentos relacionados con la solución propuesta y con las herramientas que se utilizarán durante su construcción.

API REST

Una API permite que dos componentes de software intercambien información a partir de reglas previamente definidas. En una aplicación web, esta comunicación hace posible que la interfaz solicite datos o envíe operaciones al servidor sin acceder directamente a la lógica interna ni a la base de datos. Dentro de este enfoque, IBM (s. f.) señala que “una API REST es una interfaz de programación de aplicaciones (API) que se ajusta a los principios de diseño” del estilo arquitectónico REST (sección “¿Qué es una API REST?”, párr. 1). Este tipo de interfaz organiza las operaciones alrededor de los recursos que forman parte de una aplicación y establece una manera uniforme de acceder a ellos.

En el proyecto, la API REST se encargará de procesar las solicitudes relacionadas con usuarios, aeronaves, vuelos, asientos y reservas. La interfaz web enviará cada operación al endpoint correspondiente, mientras que el servidor validará los datos y aplicará las reglas de negocio antes de generar una respuesta. Esta separación permitirá trabajar los módulos de forma ordenada y evitará que la lógica principal quede incorporada directamente en las pantallas.

Protocolo HTTP

El intercambio de información con la API se realizará mediante HTTP. Mozilla (2025) lo define como “un protocolo de la capa de aplicación para la transmisión de documentos hipermedia,

como HTML” (párr. 1). Aunque fue diseñado inicialmente para la comunicación entre navegadores y servidores, también se utiliza para el consumo de servicios web y el intercambio de datos entre aplicaciones. Las solicitudes del sistema utilizarán métodos HTTP de acuerdo con la acción que deba ejecutarse. La consulta de información no tendrá el mismo tratamiento que la creación de una reserva o la actualización de un usuario. El servidor deberá responder con un resultado comprensible, indicando si la operación fue realizada correctamente o si existe alguna condición que impida completarla.

Los datos se intercambiarán en formato JSON, lo que permitirá conservar una estructura común entre el frontend y el backend. Esta uniformidad será necesaria para que todos los módulos interpreten de la misma manera la información recibida y los posibles mensajes de error.

Base de datos relacional

La información del proyecto se almacenará en una base de datos relacional. IBM (s. f.) explica que “una base de datos relacional es un tipo de base de datos que organiza los datos en filas y columnas” (sección “¿Qué es una base de datos relacionales?”, párr. 1). Esta organización permite distribuir la información en tablas y establecer relaciones mediante identificadores comunes. Este modelo responde a la naturaleza del sistema, debido a que sus registros dependen unos de otros. Una reserva debe asociarse con el usuario que la realiza, el vuelo seleccionado y el asiento asignado. El vuelo, por su parte, se relaciona con una ruta y una aeronave. Estas dependencias deberán reflejarse mediante claves primarias y foráneas para conservar la conexión entre los datos.

La base de datos también deberá aplicar restricciones que respalden las reglas del sistema. No será suficiente con ocultar un asiento ocupado en la interfaz, ya que la disponibilidad deberá comprobarse antes de guardar una reserva. De esta forma, la integridad no dependerá únicamente de lo que observa el usuario, sino de las validaciones realizadas al procesar la operación.

Autenticación y segundo factor

La autenticación permite verificar las credenciales de una cuenta antes de conceder acceso a las funciones del sistema. En el proyecto, el proceso comenzará con el correo o nombre de usuario y la contraseña registrados. Una vez comprobados estos datos, se solicitará un código OTP como segundo paso de validación. Microsoft (s. f.) indica que “la autenticación multifactor (MFA)

es un proceso de seguridad que requiere más de una forma de comprobación para confirmar la identidad” (sección “Definición de la autenticación multifactor”, párr. 1). Esto reduce la dependencia de una sola credencial y agrega una verificación adicional al inicio de sesión.

La implementación del proyecto tendrá un alcance académico. El código OTP será temporal y deberá verificarse antes de permitir el ingreso, pero no se pretende reproducir una infraestructura empresarial de identidad. Su finalidad será representar el flujo de una autenticación reforzada y comprobar que el usuario complete ambas etapas antes de recibir acceso.

Control de acceso basado en roles

Superar la autenticación no significa que todos los usuarios puedan ejecutar las mismas operaciones. El acceso debe limitarse según las responsabilidades asignadas a cada perfil. Microsoft (2026) señala que los roles permiten “conceder permisos pormenorizados a los administradores, que cumplen el principio de privilegios mínimos” (párr. 2). Este principio consiste en otorgar únicamente los permisos necesarios para cumplir una función determinada. El sistema contará con los roles de administrador y cliente o pasajero. El administrador podrá gestionar la información operativa y consultar registros generales, mientras que el cliente tendrá acceso a las funciones asociadas con sus propias reservas.

La asignación de permisos deberá validarse desde el backend para impedir que un usuario consuma directamente un endpoint que no corresponde a su rol. La interfaz mostrará las opciones disponibles según el perfil autenticado, pero esta adaptación visual será complementaria. El control efectivo se realizará en el servidor antes de procesar cada solicitud protegida.

Gestión de vuelos, asientos y reservas

El vuelo programado representa la unidad sobre la cual se efectuará una reserva. Cada registro deberá indicar la ruta, la fecha, la hora, la aeronave asignada, el precio y el estado operativo. La disponibilidad dependerá tanto de la condición del vuelo como de los asientos que todavía no se encuentren asociados con una reserva activa. La selección de asiento se presentará mediante una cuadrícula sencilla. Esta representación permitirá identificar espacios disponibles y ocupados sin desarrollar un mapa realista de aeronave.

Una vez seleccionado el asiento, el servidor tendrá que comprobar nuevamente su estado antes de crear la reserva, porque la información mostrada en la pantalla puede cambiar durante el

proceso. La reserva seguirá un ciclo definido mediante estados. Podrá permanecer pendiente mientras se completa el pago simulado, pasar a confirmada cuando este sea aprobado o cambiar a cancelada cuando corresponda. Cada transición deberá producir el ajuste necesario sobre el asiento para evitar diferencias entre la disponibilidad registrada y la presentada al usuario.

Bitácora y trazabilidad

La bitácora permitirá conservar evidencia de las acciones relevantes ejecutadas dentro del sistema. Su contenido no se limitará a errores técnicos, ya que también registrará operaciones funcionales que deben poder rastrearse posteriormente. Microsoft (2026) explica que el registro estructurado ayuda a “supervisar el comportamiento de la aplicación y diagnosticar problemas” (párr. 1). Esto permite revisar los eventos producidos durante la ejecución y reconocer información útil para analizar situaciones que no generaron el resultado esperado.

Cada entrada de la bitácora deberá indicar el usuario relacionado, la acción realizada, la entidad afectada, la fecha y el resultado. El administrador podrá consultar esta información mediante filtros, lo que facilitará localizar eventos específicos sin revisar todos los registros almacenados. En el proyecto, la bitácora servirá como apoyo para la trazabilidad y la depuración. No se desarrollará como una solución de auditoría forense, ya que su finalidad será respaldar las operaciones principales y proporcionar evidencia durante la ejecución de las pruebas.

Notificaciones internas

Las notificaciones internas comunicarán al usuario eventos que requieran su atención dentro de la plataforma. Se mostrarán en una sección propia y podrán marcarse como leídas, sin depender de correo electrónico ni servicios externos de mensajería. Su generación estará asociada con acciones concretas, como un cambio de contraseña o una modificación relevante en el estado de una operación. El contenido deberá ser breve y permitir reconocer qué ocurrió sin consultar inmediatamente otros módulos. Este mecanismo complementará la bitácora, pero tendrá una finalidad distinta. La bitácora estará orientada al seguimiento administrativo, mientras que las notificaciones informarán directamente al usuario sobre eventos relacionados con su cuenta o sus reservas.

Documentación de la API

La documentación técnica permite conocer cómo se consume cada servicio y qué respuesta puede esperarse. IBM (2026) define OpenAPI como “un formato de descripción de API de código abierto, independiente del lenguaje de programación, legible tanto por máquinas como por humanos” (sección “OpenAPI, explicado”, párr. 1). Los endpoints del proyecto se documentarán mediante OpenAPI y Swagger, incluyendo el método HTTP, los datos requeridos, los permisos necesarios y las respuestas previstas. Esta información también podrá complementarse con una colección de Postman para ejecutar solicitudes durante el desarrollo.

La documentación deberá actualizarse junto con los servicios. Si la definición indica parámetros o respuestas diferentes de los implementados, dejará de funcionar como una guía confiable para el equipo. Por esta razón, deberá revisarse cada vez que se modifique un endpoint.

Visualización de datos

El módulo administrativo incluirá un dashboard básico para presentar información resumida. Los gráficos utilizarán datos obtenidos de la base del proyecto y podrán mostrar el estado de las reservas o la ocupación de los vuelos, según las métricas que finalmente se definan. QuickChart será utilizado como servicio externo para generar al menos una visualización. Chart.js podrá emplearse como apoyo dentro de la interfaz cuando resulte conveniente. La integración no busca construir una plataforma analítica completa, sino mostrar cómo los registros generados por el sistema pueden transformarse en información visual útil.

Los resultados gráficos deberán corresponder con las consultas realizadas al backend. No se utilizarán valores colocados únicamente para la demostración, porque esto impediría comprobar que la visualización responde al comportamiento real de la aplicación.

Pruebas y depuración

Las pruebas permiten determinar si el comportamiento del sistema coincide con los requerimientos establecidos. IBM (s. f.) explica que “las pruebas de software son el proceso de evaluar y verificar que un producto o aplicación de software funciona correctamente” (párr. 1). Esta evaluación debe realizarse sobre condiciones normales y sobre situaciones que puedan producir errores. Los casos de prueba del proyecto se concentrarán en los procesos que afectan la

seguridad y la integridad de la información. Se verificará el acceso según el rol, el funcionamiento del segundo factor, la disponibilidad de asientos y los cambios de estado de las reservas.

Cada prueba deberá definir el resultado esperado antes de su ejecución. Cuando el resultado obtenido no coincida con lo previsto, se realizará la depuración correspondiente. El equipo deberá identificar la causa, aplicar la corrección y ejecutar nuevamente el caso. La evidencia mostrará tanto el problema encontrado como el comportamiento observado después del ajuste.

Metodología

El desarrollo se llevará a cabo mediante una metodología incremental y modular, de manera que cada parte se construya sobre una base previamente revisada. El trabajo seguirá una secuencia definida que inicia con el análisis de los requerimientos, continúa con el diseño técnico y la programación, y concluye con las pruebas, la depuración y la documentación de los resultados. Este orden permitirá reducir cambios improvisados durante la implementación y mantener correspondencia entre lo planteado y lo que finalmente se entregue. La primera etapa consistirá en revisar los requerimientos funcionales y no funcionales para confirmar que cada uno se encuentre dentro del alcance establecido.

También se analizarán sus restricciones, prioridades y criterios de aceptación, con el fin de determinar qué debe realizar cada módulo y bajo qué condiciones se considerará cumplido. Cualquier ajuste necesario deberá reflejarse en el documento antes de iniciar la programación correspondiente. Una vez definidos los requerimientos, se elaborará el diseño del sistema. En esta etapa se establecerá la arquitectura general, la organización de la API, la estructura de la base de datos, los roles de usuario y las relaciones entre los módulos. Los diagramas UML, la matriz de permisos y la definición preliminar de endpoints servirán como guía para representar el comportamiento esperado y evitar diferencias de interpretación entre los integrantes.

La programación se realizará por módulos relacionados entre sí. Primero se construirá la base de seguridad y administración de usuarios; después se incorporará la gestión operativa de aeronaves, rutas, vuelos y asientos; finalmente, se desarrollará el flujo de reservas, pagos simulados, comprobantes y reportes. Cada bloque deberá quedar funcional y documentado antes de ser entregado al integrante responsable de continuar con la siguiente parte. Las pruebas se

realizarán conforme avance la implementación y no únicamente al finalizar el sistema. Cada integrante verificará su módulo mediante casos definidos a partir de los criterios de aceptación.

Posteriormente se ejecutarán pruebas de integración para comprobar que los servicios desarrollados por separado funcionen correctamente al conectarse, especialmente en procesos como autenticación, disponibilidad de asientos, creación de reservas y actualización de estados. Cuando una prueba produzca un resultado diferente del esperado, se iniciará la etapa de depuración. El error deberá reproducirse, analizarse y corregirse antes de repetir el caso de prueba. Para facilitar este seguimiento se utilizarán los mensajes controlados de la API, los registros de la aplicación y las herramientas de depuración disponibles en el entorno de desarrollo.

La corrección no se considerará terminada hasta comprobar que el caso afectado funciona sin alterar otras operaciones. La documentación se elaborará durante todo el proceso. El informe escrito se actualizará conforme se completen el análisis, el diseño, la implementación y las pruebas. Asimismo, los endpoints se documentarán mediante Swagger u OpenAPI, junto con una colección de Postman cuando corresponda. El repositorio incluirá instrucciones de instalación, configuración y ejecución, mientras que las evidencias se organizarán según el módulo y la etapa a la que pertenezcan. GitHub se utilizará para controlar las versiones del código y conservar la trazabilidad de los aportes realizados. Cada integrante trabajará sobre las tareas asignadas y registrará sus cambios mediante commits identificables.

Antes de integrar un módulo con el resto del sistema, se revisará que el código funcione, que no afecte avances anteriores y que la documentación relacionada se encuentre actualizada. Las bitácoras del equipo complementarán este seguimiento con el detalle de las actividades desarrolladas. La distribución del trabajo seguirá un orden secuencial. El primer integrante será responsable de la base del sistema, incluyendo autenticación, roles, usuarios, cambio de contraseña, bitácora y notificaciones. El segundo continuará con aeronaves, rutas, vuelos y asientos, utilizando la estructura previamente entregada. El tercer integrante desarrollará reservas, pagos simulados, comprobantes, filtros y dashboard. Aunque cada persona tendrá un bloque principal, la revisión final, las pruebas integradas, la preparación de la presentación y la demostración serán responsabilidad conjunta del equipo.

Selección y justificación de herramientas

Para el desarrollo del sistema se seleccionó Python 3.12 junto con Django 5.2 LTS como base tecnológica. Esta combinación permite trabajar con una estructura estable y organizada, adecuada para dividir el proyecto en módulos sin convertirlo en una solución más compleja de lo necesario. Django también ofrece funciones integradas para la gestión de usuarios, protección de contraseñas, validaciones, migraciones y administración de datos, lo que reduce el tiempo destinado a construir componentes básicos desde cero. La API REST se implementó mediante Django REST Framework 3.16. Esta herramienta facilita la creación de endpoints, el procesamiento de solicitudes en formato JSON y la aplicación de permisos según el rol autenticado.

Su uso mantiene separada la lógica de negocio de la interfaz web y permite que las operaciones puedan probarse directamente desde Swagger o Postman, sin depender de las pantallas del sistema. MySQL 8.4 LTS se definió como motor de base de datos por su compatibilidad con Django y por su capacidad para manejar relaciones, restricciones e integridad entre los registros. El acceso a la información se realiza mediante el ORM de Django, de modo que las entidades y sus relaciones se representan a través de modelos de Python. Los cambios en la estructura se controlan mediante migraciones, lo que permite que los tres integrantes mantengan la misma versión de la base al actualizar el repositorio.

MySQL Workbench se utiliza para elaborar el diagrama entidad-relación y revisar visualmente las claves y cardinalidades del modelo. La interfaz se construye con Django Templates, HTML, CSS, Bootstrap, JavaScript y jQuery. Esta elección permite desarrollar las pantallas requeridas sin incorporar un framework adicional de frontend. Las acciones principales se ejecutan mediante solicitudes AJAX hacia la API, por lo que la aplicación mantiene el intercambio de información en formato JSON y demuestra el consumo real de los servicios definidos.

La autenticación se controla mediante tokens JWT utilizando Simple JWT. El segundo factor se implementa con códigos OTP temporales generados mediante PyOTP, de acuerdo con el alcance académico establecido. Los permisos se configuran desde Django REST Framework para diferenciar las funciones del administrador y del cliente o pasajero. De esta forma, las restricciones

se verifican en el servidor y no dependen únicamente de las opciones visibles en la interfaz. La documentación de los servicios se genera con drf-spectacular, OpenAPI y Swagger UI.

Esta configuración permite mantener actualizada la descripción de los endpoints, sus parámetros, respuestas y permisos. Postman se utiliza como herramienta complementaria para organizar solicitudes, ejecutar pruebas manuales y conservar evidencias del funcionamiento de la API. Las pruebas unitarias y de integración se realizan con pytest y pytest-django. Las pruebas funcionales de la interfaz se ejecutan con Selenium y Python, siguiendo una estructura semejante a la utilizada en el laboratorio desarrollado durante el curso. La depuración se apoya en el depurador de Cursor o Visual Studio Code y en los registros generados por Django, con el propósito de identificar la causa de los errores y comprobar las correcciones aplicadas.

Docker Compose se utiliza para definir el entorno común del equipo. La configuración incluye la aplicación Django, la base de datos MySQL y el servicio de Selenium, por lo que cada integrante puede ejecutar las mismas versiones y dependencias desde su computadora. GitHub mantiene el código, las migraciones, los datos semilla, las pruebas y la documentación, mientras que PlantUML permite conservar los diagramas UML como archivos editables dentro del repositorio. Para cumplir con la integración de una API externa, el sistema utiliza QuickChart en al menos una visualización del dashboard administrativo. Chart.js puede emplearse como complemento para gráficos ejecutados directamente en la interfaz. Los datos mostrados se obtienen de la base del proyecto y reflejan el comportamiento real de las reservas y los vuelos registrados.

Diseño del sistema

El diseño del sistema traduce los requerimientos definidos en una estructura técnica que servirá como guía durante la programación. En esta etapa se establece cómo se comunicarán los componentes, qué información almacenará la base de datos, cuáles serán los permisos de cada rol y de qué manera se organizarán los servicios de la API. Las decisiones planteadas responden al alcance funcional acordado para la gestión de vuelos y reservas.

Arquitectura general

La arquitectura del sistema se definió bajo un modelo cliente-servidor organizado en capas. La interfaz web constituye la capa de presentación y se construye mediante plantillas de Django, Bootstrap y JavaScript. Desde esta interfaz, los usuarios realizan solicitudes AJAX hacia los

endpoints de la API, sin acceder directamente a la base de datos ni ejecutar reglas de negocio desde el navegador. La capa de aplicación está formada por Django y Django REST Framework. Las rutas reciben las solicitudes HTTP y las dirigen hacia las vistas o ViewSets correspondientes. Los serializers se encargan de validar y transformar los datos de entrada y salida, mientras que los servicios contienen las reglas necesarias para procesar autenticaciones, reservas, pagos, cancelaciones y cambios de estado.

El acceso a la información se realiza mediante los modelos y el ORM de Django. Esta capa traduce las operaciones del sistema en consultas sobre MySQL y conserva las relaciones definidas entre usuarios, vuelos, asientos, reservas y pagos. La interfaz web no cuenta con acceso directo a las tablas, por lo que toda consulta o modificación debe pasar por la API. La seguridad se aplica como un componente transversal. Simple JWT controla los tokens de acceso, PyOTP gestiona los códigos temporales del segundo factor y los permisos de Django REST Framework restringen los endpoints según el rol. Antes de procesar una operación protegida, el servidor verifica la identidad del usuario, el estado de su cuenta y la autorización necesaria.

La bitácora y las notificaciones también se integran de forma transversal. Los módulos pueden registrar eventos relevantes después de completar una acción y generar avisos cuando el usuario deba conocer algún cambio. Esta estructura permite reutilizar ambos componentes sin duplicar su lógica en cada módulo. La documentación de la API se genera mediante drf-spectacular y se presenta en Swagger UI. QuickChart recibe los datos preparados por el módulo de reportes y devuelve la visualización utilizada en el dashboard. Estas integraciones no acceden directamente a la base, sino que dependen de la información procesada por el backend.

El entorno completo se ejecuta mediante Docker Compose. La aplicación Django y MySQL funcionan en contenedores separados dentro de una misma red, mientras que Selenium se utiliza durante las pruebas funcionales. GitHub conserva los archivos necesarios para que cada integrante pueda levantar la solución y trabajar sobre una configuración equivalente.

Tabla 1.
Arquitectura del Sistema

Componente	Tecnología utilizada	Responsabilidad
------------	----------------------	-----------------

Interfaz web	Django Templates, HTML, CSS, Bootstrap, JavaScript y jQuery	Presentar formularios, consultas, asientos, reservas, notificaciones, comprobantes y reportes según el rol autenticado.
Comunicación cliente-servidor	AJAX, JSON y HTTP	Enviar solicitudes desde la interfaz hacia la API y procesar las respuestas recibidas.
API REST	Django REST Framework	Exponer los endpoints, recibir las solicitudes, comprobar permisos y devolver respuestas estructuradas en formato JSON.
Enrutamiento	URLs, routers y ViewSets de Django REST Framework	Asociar cada ruta de la API con la operación encargada de atenderla.
Validación y serialización	Serializers de Django REST Framework	Validar los datos de entrada y transformar los modelos en respuestas JSON.
Lógica de negocio	Servicios y validadores de Django	Aplicar las reglas relacionadas con usuarios, vuelos, disponibilidad, reservas, pagos y cancelaciones.
Autenticación	Simple JWT	Emitir, validar y renovar los tokens utilizados para acceder a los endpoints protegidos.
Segundo factor	PyOTP	Generar y comprobar códigos OTP temporales durante el inicio de sesión.
Control de acceso	Permisos de Django REST Framework	Restringir las operaciones según el rol de administrador o cliente/pasajero.
Acceso a datos	ORM de Django	Ejecutar consultas, registrar cambios y manejar relaciones sin exponer directamente la base de datos.

Base de datos	MySQL 8.4 LTS	Almacenar la información y conservar la integridad entre los registros del sistema.
Migraciones	Sistema de migraciones de Django	Controlar los cambios en la estructura de la base de datos y mantenerla sincronizada entre los integrantes.
Bitácora	Aplicación de auditoría en Django	Registrar acciones relevantes, usuarios involucrados, entidades afectadas y resultados obtenidos.
Notificaciones	Aplicación de notificaciones en Django	Generar y mostrar avisos relacionados con cuentas, reservas y cambios de estado.
Documentación de la API	drf-spectacular, OpenAPI y Swagger UI	Documentar los endpoints, parámetros, respuestas, códigos HTTP y permisos requeridos.
Pruebas de API	pytest, pytest-django y Postman	Verificar reglas, endpoints, integración con la base de datos y respuestas del servidor.
Pruebas funcionales	Selenium con Python	Automatizar flujos realizados desde la interfaz y generar evidencias de ejecución.
Reportes	QuickChart y Chart.js	Presentar información gráfica obtenida de los datos reales del sistema.
Contenedores	Docker Compose	Ejecutar la aplicación, MySQL y Selenium bajo una configuración común para todo el equipo.
Control de versiones	GitHub	Registrar cambios, administrar ramas, integrar módulos y conservar la trazabilidad de los aportes.

Modelado y diagramas	MySQL Workbench y PlantUML	Representar el modelo entidad-relación, los casos de uso, las clases y los diagramas de secuencia.
----------------------	----------------------------	--

Fuente: Elaboración Propia.

Roles del sistema

El sistema contará con dos roles principales: administrador y cliente o pasajero. El rol se asociará con cada usuario y será validado en las operaciones protegidas de la API. El administrador tendrá a cargo la configuración y supervisión general. Podrá gestionar usuarios, aeronaves, rutas y vuelos, además de consultar reservas, bitácoras y reportes. También tendrá autorización para cancelar vuelos cuando sea necesario y revisar la información operativa registrada. El cliente o pasajero utilizará las funciones relacionadas con el proceso de reserva.

Podrá buscar vuelos, revisar la disponibilidad de asientos, registrar una reserva, realizar el pago simulado, consultar su comprobante y cancelar una operación propia mientras las reglas del sistema lo permitan. No tendrá acceso a información perteneciente a otros usuarios ni a módulos administrativos. Ambos roles podrán cambiar su propia contraseña, consultar sus notificaciones y completar el proceso de autenticación definido. La autorización se comprobará en el backend antes de procesar cada solicitud, incluso cuando la opción correspondiente no se muestre en la interfaz.

Matriz de permisos

La matriz de permisos establece las operaciones permitidas para cada rol y servirá como referencia para configurar las restricciones de los endpoints.

Tabla 2.

Matriz de Permisos

Funcionalidad	Administrador	Cliente o pasajero
Iniciar sesión y completar el segundo factor	Sí	Sí
Cambiar su propia contraseña	Sí	Sí
Consultar sus notificaciones	Sí	Sí

Marcar sus notificaciones como leídas	Sí	Sí
Crear usuarios	Sí	No
Consultar el listado de usuarios	Sí	No
Actualizar datos de usuarios	Sí	No
Desactivar usuarios	Sí	No
Gestionar aeronaves	Sí	No
Gestionar rutas aéreas	Sí	No
Programar y actualizar vuelos	Sí	No
Cancelar administrativamente un vuelo	Sí	No
Consultar vuelos disponibles	Sí	Sí
Consultar los asientos de un vuelo	Sí	Sí
Crear una reserva	No	Sí
Consultar sus propias reservas	No	Sí
Consultar todas las reservas	Sí	No
Filtrar reservas por vuelo, fecha o estado	Sí	No
Realizar un pago simulado	No	Sí
Consultar su comprobante	No	Sí
Cancelar una reserva propia	No	Sí
Consultar la bitácora	Sí	No
Aplicar filtros sobre la bitácora	Sí	No
Consultar el dashboard administrativo	Sí	No

Fuente: Elaboración Propia.

Modelo de base de datos

La base de datos se diseñó mediante un modelo relacional, ya que la información del sistema mantiene dependencias que deben conservarse durante todo el flujo de operación. Los usuarios, vuelos, asientos, reservas y pagos no se gestionan como registros aislados, sino como entidades relacionadas que permiten identificar el origen y el estado de cada transacción. Las entidades se implementaron mediante modelos de Django. Cada modelo representa una estructura de información del sistema e incluye los atributos, restricciones y relaciones necesarios para su funcionamiento. Esta organización permite mantener separados los componentes de seguridad, la gestión operativa de vuelos y el proceso transaccional de reservas.

Las relaciones se definieron por medio del ORM de Django, utilizando asociaciones acordes con la naturaleza de los datos. De esta manera, una reserva queda vinculada con el usuario que la realizó, el vuelo seleccionado y el asiento asignado, mientras que cada vuelo mantiene relación con una ruta y una aeronave. El acceso a estos registros se realiza desde la aplicación, sin que la interfaz web consulte directamente las tablas de MySQL. Los cambios en la estructura se administraron mediante las migraciones de Django. Este mecanismo permite registrar de forma ordenada la creación o modificación de modelos y reproducir los mismos cambios en los entornos de los integrantes.

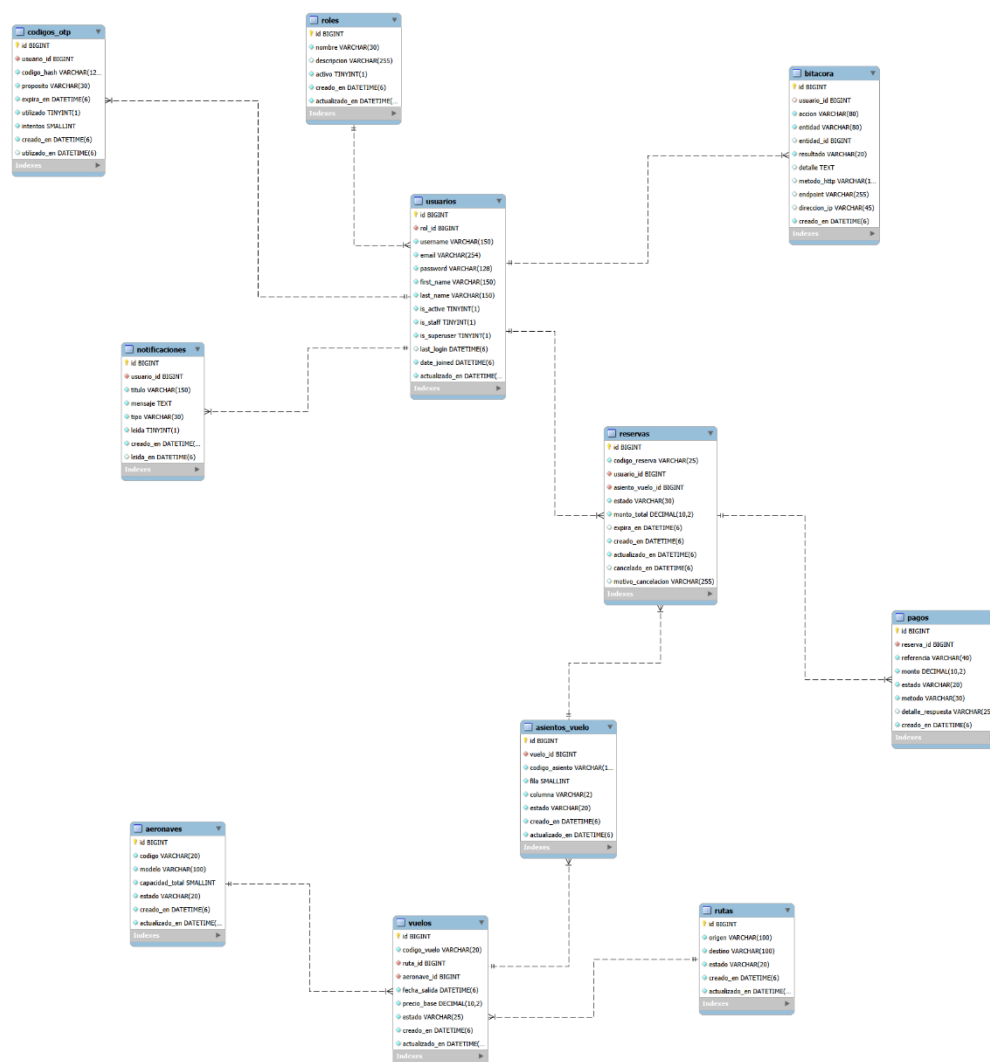
Con ello se evita que cada persona trabaje con una versión distinta de la base de datos o tenga que aplicar ajustes manuales sin evidencia dentro del repositorio. El diseño visual se representó en MySQL Workbench mediante un diagrama entidad-relación. Esta herramienta permitió revisar las entidades, sus claves y las cardinalidades establecidas antes de consolidar el modelo utilizado por la aplicación. La representación gráfica mantiene correspondencia con los modelos definidos en Django y sirve como referencia para comprender la estructura general de los datos. La base de datos MySQL se ejecuta en un contenedor administrado mediante Docker Compose. Cada integrante dispone de una instancia local con la misma configuración, mientras que las migraciones y los datos semilla se comparten a través de GitHub.

Esta forma de trabajo permite reproducir el entorno sin compartir directamente archivos internos de la base ni depender de configuraciones particulares de cada computadora. El modelo incorpora restricciones orientadas a conservar la integridad de la información. El correo de cada usuario debe ser único, una ruta no puede registrar el mismo origen y destino, y un asiento no

puede mantenerse asociado con más de una reserva activa dentro del mismo vuelo. También se utilizan estados para preservar el historial de las operaciones sin eliminar registros que resulten necesarios para la trazabilidad.

Ilustración 1.

Modelo entidad-relación del sistema web de gestión de vuelos y reservas



Fuente: Elaboración Propia.

Diagramas UML

El diseño del sistema se representó mediante diagramas UML, con el propósito de documentar sus funcionalidades, participantes y procesos principales antes de completar la implementación. Cada representación responde a una perspectiva diferente y mantiene relación

con los requerimientos, la matriz de permisos, el modelo de base de datos y la arquitectura definida para la solución. Los diagramas se elaboraron mediante PlantUML, una herramienta que permite describir las estructuras mediante código y generar representaciones visuales de forma automática. Los archivos editables se conservaron en formato .puml dentro del repositorio, mientras que las imágenes exportadas se incorporaron al informe como evidencia del diseño realizado.

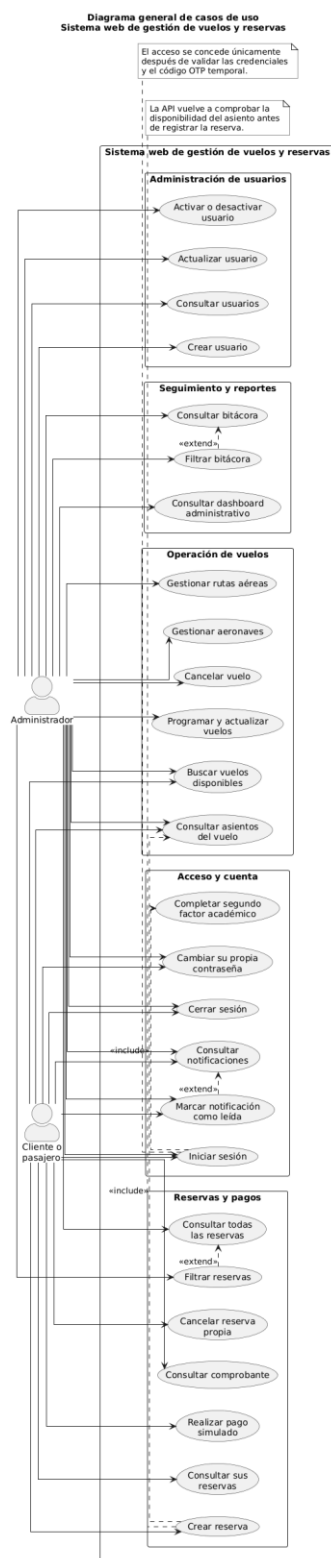
Diagrama general de casos de uso

El diagrama general de casos de uso identifica al administrador y al cliente o pasajero como actores principales. Ambos pueden autenticarse, completar el segundo factor, cerrar sesión, cambiar su contraseña y consultar sus notificaciones. Sin embargo, las demás funciones se encuentran delimitadas por los permisos establecidos para cada rol. El administrador puede gestionar usuarios, aeronaves, rutas y vuelos, además de cancelar vuelos, consultar reservas generales, revisar la bitácora y acceder al dashboard. El cliente puede buscar vuelos, consultar la disponibilidad de asientos, crear una reserva, realizar el pago simulado, obtener su comprobante y cancelar sus propias operaciones cuando las reglas del sistema lo permitan.

La relación de inclusión entre el inicio de sesión y la verificación OTP indica que el segundo factor forma parte obligatoria del proceso de acceso. Las relaciones de extensión se utilizaron en las funciones de filtrado y lectura de notificaciones, debido a que complementan una consulta principal, pero no se ejecutan necesariamente en todos los casos.

Ilustración 2.

Diagrama general de casos de uso del sistema



Fuente: Elaboración Propia.

Fuente: Elaboración Propia.

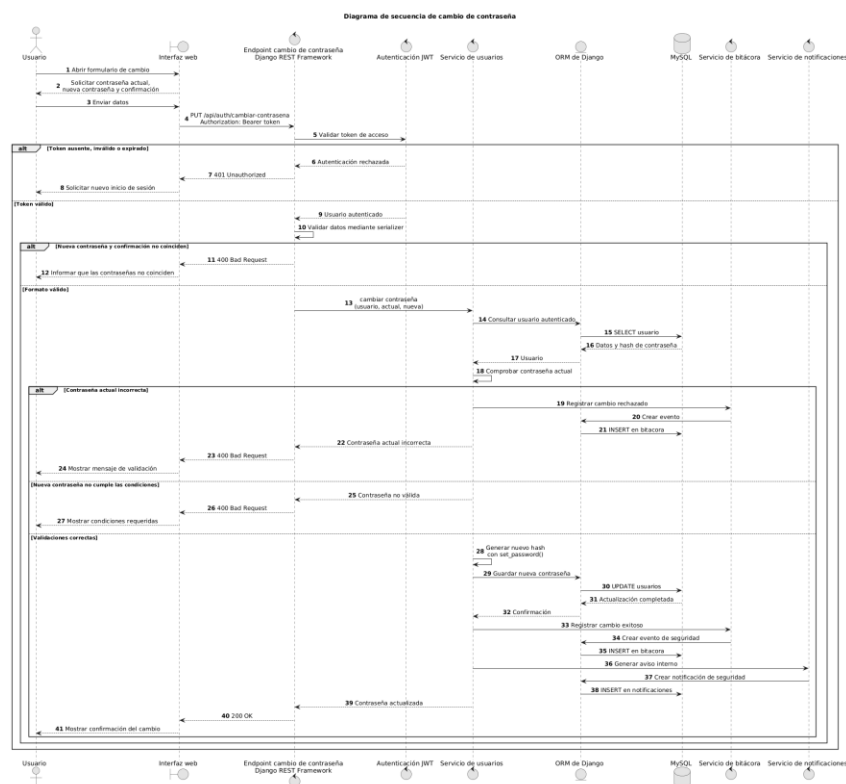
Diagrama de secuencia de cambio de contraseña

El cambio de contraseña se encuentra disponible únicamente para usuarios autenticados. La interfaz envía la solicitud junto con el token JWT, el cual se valida antes de procesar los datos. Si el token no existe, se encuentra vencido o no es válido, la API rechaza la operación y solicita un nuevo inicio de sesión. Después de autenticar al usuario, el sistema compara la contraseña actual con el hash almacenado y comprueba que la nueva credencial coincida con su confirmación y cumpla las condiciones definidas. La actualización se ejecuta mediante los mecanismos de seguridad de Django, evitando almacenar contraseñas en texto plano.

Una vez completado el cambio, se registra el evento en la bitácora y se crea una notificación interna de seguridad. Este aviso permite que el usuario conozca la modificación realizada sobre su cuenta. El diagrama también representa las respuestas generadas cuando la contraseña actual es incorrecta o la nueva credencial no cumple las validaciones requeridas.

Ilustración 4.

Diagrama de secuencia de cambio de contraseña



Endpoints principales

Los servicios principales de la API se definieron de acuerdo con los recursos, procesos y permisos establecidos durante el diseño del sistema. Las rutas utilizan el prefijo /api/ y se organizan por módulo para facilitar su implementación, documentación y posterior ejecución mediante Swagger y Postman. Las solicitudes y respuestas intercambian información en formato JSON. Los endpoints protegidos requieren un token JWT válido y comprueban el rol del usuario antes de procesar la operación. Además, las funciones relacionadas con reservas, comprobantes, pagos y notificaciones verifican que el registro consultado pertenezca al usuario autenticado, salvo cuando la operación sea realizada por un administrador autorizado.

Las respuestas de la API mantienen una estructura uniforme que permite identificar si la operación fue satisfactoria, el mensaje asociado y los datos devueltos.

```
{
  "success": true,
  "message": "Operación realizada correctamente.",
  "data": {}
}
```

Cuando se produce un error, la respuesta indica que la operación no pudo completarse y detalla las validaciones que impidieron procesarla.

```
{
  "success": false,
  "message": "No fue posible completar la operación.",
  "errors": {}
}
```

Los principales códigos HTTP utilizados son 200 OK para consultas o actualizaciones satisfactorias, 201 Created para registros creados, 204 No Content cuando no es necesario devolver información, 400 Bad Request para datos inválidos, 401 Unauthorized para solicitudes sin autenticación válida, 403 Forbidden para permisos insuficientes, 404 Not Found cuando el recurso no existe y 409 Conflict cuando la operación produce un conflicto con el estado actual de la información.

Tabla 3.
Autenticación y seguridad

Método	Endpoint	Descripción	Acceso
POST	/api/auth/login/	Valida el correo y la contraseña. Cuando	Público

		las credenciales son correctas, genera el desafío para continuar con el segundo factor.	
POST	/api/auth/verificar-otp/	Comprueba el código OTP temporal, su vigencia, los intentos y su estado de utilización. Si es correcto, genera los tokens JWT.	Público con identificador temporal
POST	/api/auth/refresh/	Genera un nuevo token de acceso a partir de un refresh token válido.	Usuario con refresh token
POST	/api/auth/logout/	Invalida el refresh token y finaliza la sesión.	Administrador y cliente
PUT	/api/auth/cambiar-contrasena/	Comprueba la contraseña actual, valida la nueva credencial, actualiza el hash, registra el evento y genera una notificación.	Administrador y cliente

Fuente: Elaboración Propia.

Tabla 4.
Usuarios

Método	Endpoint	Descripción	Acceso
GET	/api/usuarios/	Devuelve el listado de usuarios y admite filtros por rol, estado, nombre o correo.	Administrador
POST	/api/usuarios/	Crea una cuenta y le asigna uno de los roles definidos.	Administrador
GET	/api/usuarios/{id}/	Consulta la información de un usuario específico.	Administrador
PUT	/api/usuarios/{id}/	Sustituye los datos editables de un usuario.	Administrador
PATCH	/api/usuarios/{id}/	Actualiza parcialmente los datos de un usuario.	Administrador
PATCH	/api/usuarios/{id}/estado/	Activa o desactiva una cuenta sin eliminar su historial.	Administrador
GET	/api/usuarios/perfil/	Devuelve la información básica del usuario autenticado y su rol.	Administrador y cliente

Fuente: Elaboración Propia.

Tabla 5.
Aeronaves

Método	Endpoint	Descripción	Acceso
--------	----------	-------------	--------

GET	/api/aeronaves/	Consulta las aeronaves y permite filtrar por código, modelo o estado.	Administrador
POST	/api/aeronaves/	Registra una aeronave con código, modelo, capacidad y estado.	Administrador
GET	/api/aeronaves/{id}/	Consulta los datos de una aeronave determinada.	Administrador
PUT	/api/aeronaves/{id}/	Actualiza completamente la información de la aeronave.	Administrador
PATCH	/api/aeronaves/{id}/	Modifica parcialmente la información de la aeronave.	Administrador
PATCH	/api/aeronaves/{id}/estado/	Activa o desactiva la aeronave sin eliminar los vuelos históricos relacionados.	Administrador

Fuente: Elaboración Propia.

Tabla 6.
Rutas aéreas

Método	Endpoint	Descripción	Acceso
--------	----------	-------------	--------

GET	/api/rutas/	Consulta las rutas y permite filtrar por origen, destino o estado.	Administrador
POST	/api/rutas/	Registra una ruta aérea y comprueba que el origen y el destino sean diferentes.	Administrador
GET	/api/rutas/{id}/	Consulta una ruta específica.	Administrador
PUT	/api/rutas/{id}/	Actualiza completamente los datos de una ruta.	Administrador
PATCH	/api/rutas/{id}/	Modifica parcialmente una ruta.	Administrador
PATCH	/api/rutas/{id}/estado/	Activa o desactiva una ruta sin eliminar su historial.	Administrador

Fuente: Elaboración Propia.

Tabla 7.
Vuelos y asientos

Método	Endpoint	Descripción	Acceso
GET	/api/vuelos/	Consulta los vuelos registrados y permite aplicar filtros administrativos.	Administrador

POST	/api/vuelos/	Programa un vuelo asociado con una ruta y una aeronave activas.	Administrador
GET	/api/vuelos/{id}/	Consulta los datos de un vuelo específico.	Administrador y cliente
PUT	/api/vuelos/{id}/	Actualiza completamente los datos permitidos de un vuelo.	Administrador
PATCH	/api/vuelos/{id}/	Modifica parcialmente un vuelo y respeta sus reglas de estado.	Administrador
GET	/api/vuelos/buscar/	Busca vuelos disponibles por origen, destino y fecha, excluyendo los cerrados, cancelados o sin asientos libres.	Administrador y cliente
PATCH	/api/vuelos/{id}/estado/	Cambia el estado entre programado, disponible o cerrado cuando la transición es válida.	Administrador
PATCH	/api/vuelos/{id}/cancelar/	Cancela el vuelo, actualiza las	Administrador

		reservas afectadas y genera bitácoras y notificaciones.	
GET	/api/vuelos/{id}/asientos/	Devuelve la cuadrícula de asientos del vuelo con su código y estado.	Administrador y cliente
POST	/api/vuelos/{id}/asientos/generar/	Genera los asientos del vuelo según la capacidad de la aeronave.	Administrador
PATCH	/api/asientos/{id}/estado/	Modifica el estado de un asiento cuando existe una operación administrativa autorizada.	Administrador

Fuente: Elaboración Propia.

Tabla 8.
Reservas

Método	Endpoint	Descripción	Acceso
POST	/api/reservas/	Crea una reserva pendiente después de comprobar nuevamente la disponibilidad del vuelo y del asiento.	Cliente

GET	/api/reservas/mis-reservas/	Consulta las reservas pertenecientes al usuario autenticado.	Cliente
GET	/api/reservas/	Consulta todas las reservas y permite filtros por usuario, vuelo, fecha, estado o código.	Administrador
GET	/api/reservas/{id}/	Consulta el detalle de una reserva. El cliente solo puede consultar una operación propia.	Administrador o propietario
PATCH	/api/reservas/{id}/cancelar/	Cancela una reserva propia cuando su estado y las reglas del sistema lo permiten.	Cliente
GET	/api/reservas/{id}/comprobante/	Genera el comprobante tipo boleto con los datos de la reserva, el vuelo, el asiento y el pago aprobado.	Administrador o propietario

Fuente: Elaboración Propia.

Tabla 9.
Pagos simulados

Método	Endpoint	Descripción	Acceso
POST	/api/reservas/{id}/pagos/	Ejecuta la simulación de pago de una reserva pendiente y registra el resultado como aprobado o rechazado.	Cliente propietario
GET	/api/reservas/{id}/pagos/	Consulta los intentos de pago asociados con una reserva.	Administrador o propietario
GET	/api/pagos/{id}/	Consulta el detalle de un pago simulado específico.	Administrador o propietario

Fuente: Elaboración Propia.

Tabla 10.
Bitácora

Método	Endpoint	Descripción	Acceso
GET	/api/bitacora/	Consulta los eventos registrados y admite filtros por usuario, acción, entidad, resultado y rango de fechas.	Administrador
GET	/api/bitacora/{id}/	Consulta el detalle de un evento específico.	Administrador

Fuente: Elaboración Propia.

Tabla 11.
Notificaciones

Método	Endpoint	Descripción	Acceso
GET	/api/notificaciones/	Consulta únicamente las notificaciones del usuario autenticado y permite filtrar por estado de lectura o tipo.	Administrador y cliente
GET	/api/notificaciones/{id}/	Consulta el detalle de una notificación propia.	Propietario
PATCH	/api/notificaciones/{id}/leida/	Marca una notificación específica como leída y registra la fecha correspondiente.	Propietario
PATCH	/api/notificaciones/marcar-todas-leidas/	Marca como leídas todas las notificaciones pendientes del usuario autenticado.	Administrador y cliente

Fuente: Elaboración Propia.

Tabla 12.
Reportes y dashboard

Método	Endpoint	Descripción	Acceso
GET	/api/reportes/resumen/	Devuelve los indicadores generales	Administrador

		utilizados en el dashboard administrativo.	
GET	/api/reportes/reservas-por-estado/	Devuelve la cantidad de reservas agrupadas por estado.	Administrador
GET	/api/reportes/ocupacion-vuelos/	Calcula los asientos disponibles y reservados de los vuelos consultados.	Administrador
GET	/api/reportes/grafico-quickchart/	Prepara los datos y genera la visualización mediante la API de QuickChart.	Administrador

Fuente: Elaboración Propia.

Descripción de módulos

El sistema se organizó en aplicaciones internas de Django que agrupan funcionalidades relacionadas. Esta división no representa una arquitectura de microservicios, ya que todos los componentes forman parte de una misma solución, comparten la configuración general y utilizan la misma base de datos. La organización modular permite distribuir el trabajo entre los integrantes, reducir dependencias innecesarias y mantener separadas las responsabilidades de cada bloque. Cada aplicación puede contener sus modelos, serializers, permisos, servicios, vistas, rutas y pruebas. Las operaciones que involucran más de un módulo se coordinan mediante servicios y transacciones para evitar que la lógica quede duplicada o distribuida de forma inconsistente.

Módulo de autenticación

El módulo de autenticación controla el proceso mediante el cual los usuarios obtienen acceso al sistema. Su primera responsabilidad consiste en validar el correo y la contraseña

mediante los mecanismos de seguridad de Django. También comprueba que la cuenta se encuentre activa antes de iniciar el segundo factor. Cuando las credenciales son correctas, el módulo genera un código OTP académico temporal, almacena su hash y establece una fecha de expiración. El acceso solo se completa después de verificar correctamente este código. En ese momento, Simple JWT genera los tokens necesarios para consumir los endpoints protegidos.

El módulo también administra la renovación de tokens, el cierre de sesión y el cambio de contraseña. Las operaciones relevantes se comunican con la bitácora y las notificaciones para conservar evidencia del acceso, los intentos fallidos y las modificaciones de seguridad. Esta funcionalidad se implementa en la aplicación interna autenticacion.

Módulo de Usuarios

El módulo de usuarios gestiona las cuentas, los roles y el estado de acceso. El modelo personalizado de Django almacena la información de identificación, el hash de la contraseña y los indicadores necesarios para reconocer cuentas activas, personal administrativo y superusuarios. El administrador puede crear usuarios, consultar el listado, modificar datos y cambiar el estado de una cuenta. La desactivación se utiliza en lugar de eliminar registros, debido a que un usuario puede estar relacionado con reservas, pagos, bitácoras y notificaciones que deben conservarse. El módulo también proporciona la información básica del perfil autenticado y permite determinar si la cuenta corresponde a un administrador o a un cliente. Los permisos definidos en Django REST Framework utilizan este rol para autorizar o rechazar las solicitudes. Esta funcionalidad se implementa en la aplicación interna usuarios.

Módulo de Aeronaves y Rutas

El módulo operativo administra las aeronaves y rutas necesarias para programar vuelos. Cada aeronave cuenta con código, modelo, capacidad y estado. La capacidad se utiliza posteriormente para generar los asientos del vuelo, mientras que el estado determina si puede asignarse a nuevas programaciones. Las rutas almacenan el origen, el destino y su condición operativa. El sistema impide registrar una ruta con el mismo origen y destino y evita duplicar una combinación ya existente. Tanto las aeronaves como las rutas se desactivan sin eliminarse para mantener la relación con los vuelos históricos. Estas funciones se agrupan dentro de la aplicación interna vuelos, aunque pueden organizarse en archivos y servicios separados.

Módulo de Vuelos y Asientos

El módulo de vuelos permite programar operaciones aéreas utilizando una ruta y una aeronave activas. Cada vuelo contiene un código único, fecha de salida, precio base y estado. Los estados definidos son PROGRAMADO, DISPONIBLE, CERRADO y CANCELADO. Después de programar el vuelo, el sistema genera los registros de asientos de acuerdo con la capacidad de la aeronave. Cada asiento se identifica mediante un código, una fila y una columna. Sus estados pueden ser DISPONIBLE, BLOQUEADO o RESERVADO. La búsqueda del cliente consulta únicamente vuelos disponibles que coinciden con el origen, destino y fecha solicitados. También comprueba que exista al menos un asiento libre. La cancelación administrativa de un vuelo se ejecuta mediante una transacción, debido a que debe modificar el vuelo, las reservas afectadas, los asientos, las notificaciones y la bitácora de forma coherente. Esta funcionalidad se implementa en la aplicación interna vuelos.

Módulo de Reservas

El módulo de reservas concentra el núcleo transaccional del sistema. Antes de crear una reserva, vuelve a consultar el asiento seleccionado para comprobar que continúa disponible. Esta validación se ejecuta en el servidor y no depende únicamente de lo mostrado en la interfaz. Cuando la operación se registra, se genera un código único y la reserva comienza con estado PENDIENTE. El asiento cambia temporalmente a BLOQUEADO y se establece un periodo de expiración para evitar que permanezca ocupado indefinidamente cuando el usuario no completa el pago.

La reserva puede pasar a CONFIRMADA, RECHAZADA, EXPIRADA, CANCELADA o CANCELADA_ADMIN, según el resultado de las operaciones posteriores. El módulo también permite que el cliente consulte sus propias reservas y que el administrador aplique filtros generales. La cancelación libera el asiento cuando corresponde, genera una notificación y registra el evento. El comprobante se construye a partir de la información relacionada y no necesita una tabla independiente. Esta funcionalidad se implementa en la aplicación interna reservas.

Módulo de Pagos

El módulo de pagos simula la respuesta de una operación financiera sin utilizar tarjetas, cuentas bancarias ni pasarelas reales. Cada intento se relaciona con una reserva, genera una referencia única y almacena el monto, el resultado y el detalle de la simulación. Una reserva puede tener varios intentos de pago, debido a que una primera operación puede resultar rechazada.

Cuando el pago es aprobado, el módulo actualiza la reserva a CONFIRMADA y cambia el asiento a RESERVADO. Cuando el resultado es rechazado, conserva el registro para efectos de trazabilidad y genera la notificación correspondiente. Las actualizaciones que afectan el pago, la reserva y el asiento se ejecutan dentro de una transacción para impedir estados incompletos. Esta funcionalidad se implementa en la aplicación interna pagos.

Módulo de Bitácora

El módulo de bitácora registra automáticamente las acciones relevantes realizadas dentro de la aplicación. Cada entrada puede incluir el usuario, la acción, la entidad afectada, el identificador relacionado, el resultado, el endpoint, el método HTTP, la dirección IP y la fecha. La bitácora almacena tanto operaciones satisfactorias como errores y accesos denegados. En ciertos intentos fallidos de inicio de sesión, el usuario puede permanecer vacío cuando el correo indicado no corresponde con una cuenta existente. El administrador puede consultar los registros y aplicar filtros, pero no puede modificarlos ni eliminarlos desde la interfaz. Esta restricción conserva la confiabilidad de la información utilizada durante la trazabilidad y la depuración. Esta funcionalidad se implementa en la aplicación interna auditoria.

Módulo de Notificaciones

El módulo de notificaciones informa a los usuarios sobre eventos relacionados con su cuenta, vuelos o reservas. Los avisos se generan internamente y se muestran dentro de la plataforma, sin utilizar correos electrónicos, mensajes SMS o servicios externos. Entre los eventos que pueden generar una notificación se encuentran el cambio de contraseña, la confirmación de una reserva, el rechazo de un pago, la cancelación de una reserva y la cancelación administrativa de un vuelo. Cada usuario consulta únicamente sus propias notificaciones y puede marcarlas como leídas. El módulo conserva la fecha de creación y, cuando corresponde, la fecha en que se realizó la lectura. Esta funcionalidad se implementa en la aplicación interna notificaciones.

Módulo de Reportes

El módulo de reportes consulta y agrupa los datos generados por los demás componentes para construir el dashboard administrativo. No almacena registros duplicados ni utiliza una tabla propia, debido a que sus resultados se calculan a partir de vuelos, asientos, reservas y pagos. Entre sus indicadores pueden incluirse la cantidad total de reservas, la distribución por estado, el número

de vuelos disponibles y el porcentaje de ocupación. Al menos una visualización se genera mediante QuickChart para demostrar el consumo de una API externa, mientras que Chart.js puede utilizarse como complemento en las representaciones ejecutadas desde la interfaz. Esta funcionalidad se implementa en la aplicación interna reportes.

Relación entre los Módulos

La relación entre los módulos sigue el flujo principal de la operación. El usuario se autentica y obtiene los permisos correspondientes a su rol; posteriormente consulta vuelos y asientos disponibles, crea una reserva pendiente y realiza el pago simulado. Cuando el pago es aprobado, la reserva se confirma, el asiento se marca como reservado y el sistema habilita la generación del comprobante. La bitácora y las notificaciones funcionan como componentes transversales. Los módulos de autenticación, usuarios, vuelos, reservas y pagos pueden registrar eventos en la bitácora y generar avisos internos cuando una acción requiere informar al usuario. El módulo de reportes consulta la información producida por los demás componentes, pero no modifica directamente los procesos transaccionales.

Desarrollo

Conclusiones

Recomendaciones

Referencias

Anexo A

Anexo B

Anexo C

Anexo D

Anexo E